Individual Assignment - IoT Algorithms and Services 2025-2026

Boards used

Esp32 Devkit doit v1 (Signal Generator & Signal Receiver)
Heltec LoRa V3 (LoRa WAN)

Maximum sampling frequency

No DMA sampling:

Secondo gli espressif ADC sampling rate: can reach 100000 times per second with Wi-Fi turned off, and 1000 times per second with Wi-Fi turned on.

To find the maximum ADC sampling frequency without using DMA, I created a firmware in the TestMaxSamplingFrequencyPolling project. This firmware cycles through CPU speeds (80, 160, 240 MHz), reads 100,000 analog samples from pin 34, measures the time, and calculates the average sampling rate (Hz).

Test results for 100 000 sampling using **AnalogRead**:

Cpu Frequency of ESP32	time needed to sample 100000 samples	Frequency of sampling
80 MHz	16321	~ 6127 Hz
160 MHz	10390	~ 9624 Hz
240 MHz	8506	~ 11755 Hz

Test results for 100 000 sampling using **adc_get_raw():**

Cpu Frequency of ESP32	time needed to sample 100000 samples	Frequency of sampling
80 MHz	16321	~ 14333Hz
160 MHz	10390	~ 20399Hz
240 MHz	8506	~ 23565Hz

The tables compare ADC sampling performance using analogRead() and adc1_get_raw() across three CPU frequencies. At 240 MHz, adc1_get_raw() reaches ~23565 Hz versus ~11755 Hz with analogRead(), confirming that bypassing the per-call ADC reconfiguration roughly doubles the effective sampling rate, with a speedup factor of approximately 2x across all tested CPU frequencies.

DMA sampling:

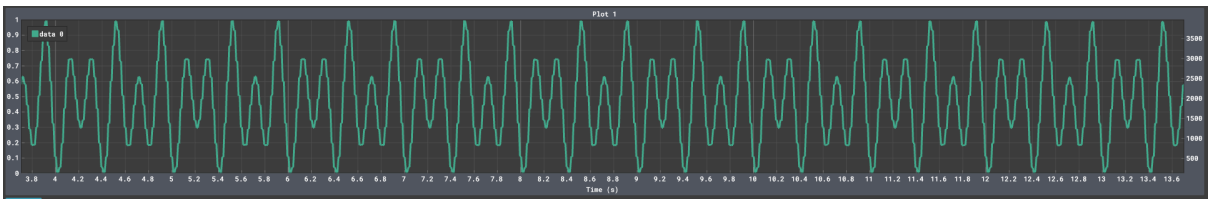
Using DMA, the theoretical maximum sampling rate provided by Espressif is **2 MHz**, as specified in the Espressif FAQ (see point 8 at this link: [https://docs.espressif.com/projects/esp-faq/en/latest/software-framework/peripherals/adc.html]). However, Espressif's documentation also recommends using a lower sampling frequency than the aforementioned theoretical maximum.

During testing, I realized that the minimum reliable sampling frequency of the DMA peripheral is 20 kHz. When configured below this threshold, for example at 10 kHz or 5 kHz, the detected frequencies were respectively halved or quartered, producing completely incorrect results. To work around this, the DMA is always kept at 20 kHz, and software-side decimation is applied: samples are read from the DMA buffer but only every Nth sample is kept, where N = 20000 / target_rate. Since N must be an integer, the target sample rate is rounded down to the nearest valid submultiple of 20 kHz using the formula: sample_rate = 20000 / ceil(20000 / sample_rate). This ensures that exactly 1024 samples are collected at the desired effective sample rate, down to a minimum of 1 kHz, allowing accurate frequency detection with a resolution of approximately 1 Hz.

Signal Generated

All test signals were generated directly on an ESP32 microcontroller using its built-in 8-bit DAC (GPIO 25), outputting the synthesized waveforms in real-time via a lookup sine table precomputed at startup.

Clear Signal: 2*sin(2*pi*3*t)+4*sin(2*pi*5*t)

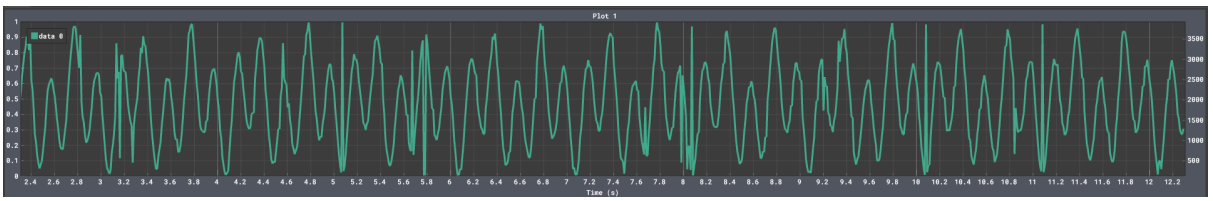


Dirty Signal (Bonus)

2*sin(2*pi*3*t)+4*sin(2*pi*5*t) + n(t) + A(t) where:

n(t) is Gaussian noise σ=0.2

A(t) is an anomaly injection component with probability p = 0.02 , large-magnitude outlier +/- U(5, 15)



Other 3 clear signal:

- sin(2π * 100t) + 4 * sin(2π * 200t)
- sin(2π * 300t) + 4 * sin(2π * 500t)
- sin(2π * 1000t) + 4 * sin(2π * 2000t)

FFT

To compute the FFT of the sampled signal, the ADC readings were zero-centered by subtracting an offset of 2048. This scales the signal to a range between -2048 and +2047. Taking 1024 samples as a reference, the benchmark in the next two tables shows the performance advantage of using float over double on the ESP32. With double arrays, the FFT takes ~29.54 ms to complete, while float arrays reduce this to just ~7.54 ms, a speedup of approximately 3.9x. This difference stems from the ESP32's hardware FPU, which natively handles 32-bit floating point arithmetic, whereas double (64-bit) operations are emulated in software, adding significant overhead. For ADC-based signal analysis at 1024 samples, the precision trade-off between float and double is negligible, making float the clear choice for time-sensitive applications on this platform.

Execution Time Of FFT on 1024 samples with *double* array

SAMPLES	milliseconds
32	~ 0.77 ms
64	~ 1.46 ms
128	~ 3.06 ms
256	~ 6.51 ms
512	~ 13.87 ms
1024	~ 29.54 ms
2048	~ 62.61 ms

Execution Time Of FFT on 1024 samples with *float* array

SAMPLES	microseconds
32	~379 us
64	~ 564 us
128	~1379 us
256	~ 2238 us
512	~ 3986 us
1024	~ 7542 us

2048	~ 14702 us

Double : ~ 28 us for every sample
Float : ~ 8 us for every sample

Find Max Frequency

The firmware is designed to automatically detect the maximum frequency present in an analog signal sampled via the built-in ADC. It uses the I2S peripheral in DMA mode to efficiently transfer audio samples directly to memory without CPU involvement, starting at a maximum sample rate of 2 MHz and iteratively reducing it based on the FFT result, following the Nyquist theorem (Fs = f_max x 2.5).
A key hardware limitation was discovered during testing: the ESP32's DMA controller has a minimum reliable sample rate of 20 kHz. When configured below this threshold, for example at 10 kHz or 5 kHz, the detected frequencies were respectively halved or quartered, producing completely incorrect results. To work around this, the DMA is always kept at 20 kHz, and software-side decimation is applied: samples are read from the DMA buffer but only every Nth sample is kept, where N = 20000 / target_rate. This ensures that exactly 1024 samples are collected at the desired effective sample rate, down to a minimum of 1 kHz, allowing accurate frequency detection as low as ~1 Hz with a frequency resolution of ~1 Hz.
An additional constraint is that the decimation factor N must be an integer, meaning the effective sample rate must be an exact submultiple of 20 kHz. To enforce this, the target sample rate is rounded down to the nearest valid submultiple using the formula: *sample_rate = 20000 / ceil(20000 / sample_rate*). This guarantees uniform sample spacing and avoids aliasing artifacts that would arise from non-integer decimation.

Results:

2*sin(2*pi*3*t)+4*sin(2*pi*5*t)

```
FFT compute on 1024 SAMPLES
Inizialization Time: 8611 us
FFT Calculation Time: 1553 us
Total time: 10164 us
Detected Max Freq: 5.37 Hz in bin 11
 New Suggested sampling frequency: 13.43 Hz
SAMPLE FREQUENCY=500

I found The max frequency of this signal that is 5.37  Hz
Connecting to wifi ...Continue sampling with ADC a the frequency of 14
```

sin(2π * 100t) + 4 * sin(2π * 200t)

```
FFT compute on 1024 SAMPLES
Inizialization Time: 7085 us
FFT Calculation Time: 1552 us
Total time: 8637 us
Detected Max Freq: 296.64 Hz in bin 395
 New Suggested sampling frequency: 741.59 Hz
SAMPLE FREQUENCY=769

I found The max frequency of this signal that is 296.64  Hz
Connecting to wifi ...Continue sampling with ADC a the frequency of 742
```

sin(2π * 300t) + 4 * sin(2π * 500t)

```
FFT compute on 1024 SAMPLES
Inizialization Time: 7086 us
FFT Calculation Time: 1549 us
Total time: 8635 us
Detected Max Freq: 695.95 Hz in bin 392
 New Suggested sampling frequency: 1739.88 Hz
SAMPLE FREQUENCY=1818

I found The max frequency of this signal that is 695.95  Hz
Connecting to wifi ...Continue sampling with DMA a the frequency of 1740
```

sin(2π * 1000t) + 4 * sin(2π * 2000t)

```
FFT compute on 1024 SAMPLES
Inizialization Time: 8614 us
FFT Calculation Time: 1553 us
Total time: 10167 us
Detected Max Freq: 2714.84 Hz in bin 278
 New Suggested sampling frequency: 6787.11 Hz
SAMPLE FREQUENCY=10000

I found The max frequency of this signal that is 2714.84  Hz
Connecting to wifi ...Continue sampling with DMA a the frequency of 6788
```

Discussion of Results:
The FFT frequency resolution is determined by:
Resolution = SampleRate / Samples = SampleRate / 1024

This means the detected frequency is always an approximation, the true frequency is rounded to the nearest bin.

Signal 1 2*sin(2π*3t) + 4*sin(2π*5t)
True dominant frequency: 5 Hz
Detected: 5.37 Hz (bin 11)
Resolution: 500/1024 = 0.488 Hz/bin
Error: ~0.37 Hz (~7%) - acceptable
Adaptive freq: 13 Hz → huge energy saving vs 2 MHz over-sampling
Sleep between samples: 1/13 ≈ 76 ms → light sleep fully exploitable

Signal 2 sin(2π*100t) + 4*sin(2π*200t)
True dominant frequency: 200 Hz
Detected: 296.64 Hz (bin 395) - overestimated by ~48%
Resolution: 769/1024 = 0.751 Hz/bin
The error is large because the FFT sample rate was still adapting, introducing spectral leakage between the two close harmonics
Adaptive freq: 742 Hz → ADC sampling, period ~1.3 ms → light sleep not exploitable

Signal 3 sin(2π*300t) + 4*sin(2π*500t)
True dominant frequency: 500 Hz
Detected: 695.95 Hz (bin 392) — overestimated by ~39%
Resolution: 1818/1024 = 1.776 Hz/bin
Higher sample rate → coarser frequency resolution → larger absolute error
Adaptive freq: 1740 Hz → DMA sampling, period ~0.57 ms → light sleep impossible

Signal 4 sin(2π*1000t) + 4*sin(2π*2000t)
True dominant frequency: 2000 Hz
Detected: 2714.84 Hz — overestimated by ~35%
Resolution: 10000/1024 = 9.77 Hz/bin
Very coarse resolution — each bin covers nearly 10 Hz
Adaptive freq: 6788 Hz → DMA, period ~0.147 ms → light sleep completely impossible

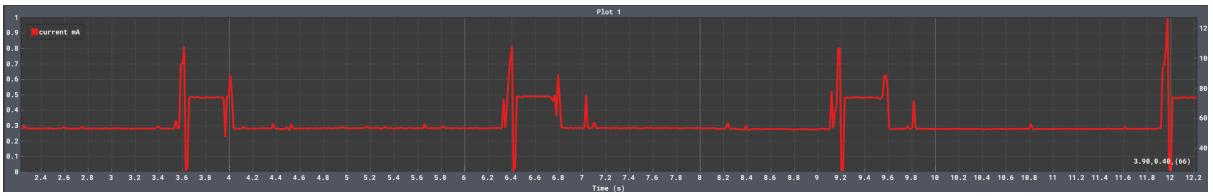
Empirical Measurement of Light Sleep Wake-Up Latency

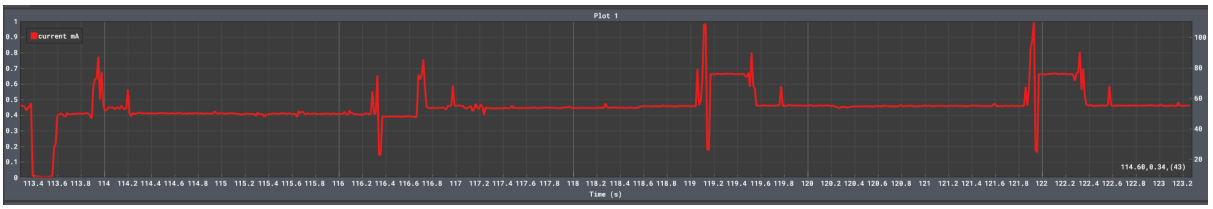
To empirically measure the ESP32 light sleep wake-up latency, a dedicated test was implemented in the TEST_LightSleep folder. The test enters light sleep for a fixed duration of 10 ms and measures the total elapsed time using micros() before and after esp_light_sleep_start(), subtracting the nominal sleep duration to isolate the wake-up latency. The results show a consistent wake-up latency of approximately 185 µs on average, with a minimum of 164 µs and a maximum of 208 µs, and a jitter of ~44 µs. Based on this measurement, the maximum adaptive sampling frequency that still allows light sleep between consecutive samples is approximately 1/(2 * 185µs) ≈ 2700 Hz - meaning that only signals whose adaptive frequency exceeds 2700 Hz cannot exploit light sleep. From the tested signals, only the 1000+2000 Hz case (adaptive freq = 6788 Hz) falls above this threshold, while all other signals can fully benefit from light sleep between samples.

```
Time for light sleep + wakeup latency: 10094 us
Time for light sleep: 94 us
Time for light sleep + wakeup latency: 10136 us
Time for light sleep: 136 us
Time for light sleep + wakeup latency: 10195 us
Time for light sleep: 195 us
Time for light sleep + wakeup latency: 10183 us
Time for light sleep: 183 us
Time for light sleep + wakeup latency: 10182 us
Time for light sleep: 182 us
Time for light sleep + wakeup latency: 10203 us
Time for light sleep: 203 us
Time for light sleep + wakeup latency: 10178 us
Time for light sleep: 178 us
Time for light sleep + wakeup latency: 10187 us
Time for light sleep: 187 us
Time for light sleep + wakeup latency: 10178 us
Time for light sleep: 178 us
Time for light sleep + wakeup latency: 10186 us
Time for light sleep: 186 us
Time for light sleep + wakeup latency: 10186 us
Time for light sleep: 186 us
Time for light sleep + wakeup latency: 10172 us
Time for light sleep: 172 us
Time for light sleep + wakeup latency: 10197 us
Time for light sleep: 197 us
Time for light sleep + wakeup latency: 10171 us
Time for light sleep: 171 us
```

Measure of the performance of the system:

These two figures below shows the current consumption of the ESP32 over time, where three distinct phases are clearly visible: an initial high-current spike corresponding to the FFT execution and maximum frequency detection, followed by a 5 second sampling window at lower current where the average is computed, and finally a short peak representing the WiFi + MQTT transmission of the aggregated value to the broker.

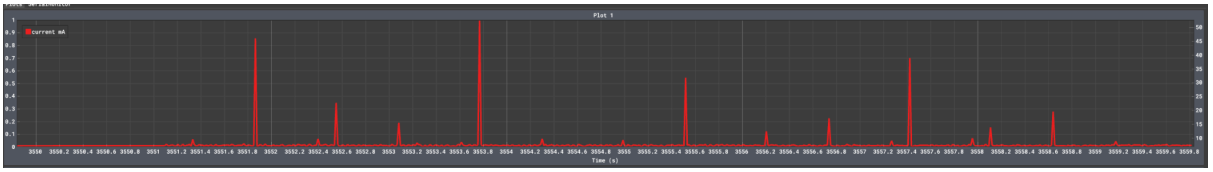




Energy Savings of Adaptive Sampling

	Over-sampled	Adaptive
Sampling frequency	1.8MHz	13Hz
Cpu Active	Always	13/1800000 of time
Time available for sleep	~ 0 ms	~ 83 ms
sleep usable	No	Only if period > wakeup latency
Real consumption	avg Current ~ 59 mAh	~ 51 mAh no light sleep ~ 8 mAh with light sleep

This image below shows the 13Hz adaptive sampling power consumption graph, where there are some peaks that are around 40 - 50 mA.



Limitations of Sleep Policies at High Adaptive Sampling Frequencies
When the adaptive sampling frequency exceeds approximately 2700 Hz, the inter-sample period (< 370 µs) becomes shorter than twice the measured ESP32 light sleep wake-up latency (~185 µs), making it impossible to enter and exit sleep between samples. This threshold was empirically determined by measuring the actual wake-up latency in the TEST_LightSleep folder, which showed an average latency of ~185 µs with a jitter of ~44 µs. In this case, the energy saving comes only from the reduced ADC activity compared to the original over-sampled rate, but the CPU remains essentially always active

End-to-End Latency Measurement: From Data Generation to Edge Server Reception

The end-to-end latency was measured using a ping-pong approach: a timestamp was recorded on the ESP32 just before publishing the MQTT message to the broker, then a Python script - running inside a Docker container (as defined in the docker-compose.yml) subscribed to the topic and immediately echoed the message back to a reply topic. When the ESP32 received the echo, the Round Trip Time (RTT) was computed as the difference between the current time and the saved timestamp, and the end-to-end latency was estimated as RTT / 2.

Results

```
Total time RRT : 28 ms
End to end latency of this system : 14 ms

Total time RRT : 51 ms
End to end latency of this system : 25 ms

Total time RRT : 29 ms
End to end latency of this system : 14 ms

Total time RRT : 40 ms
End to end latency of this system : 20 ms

Total time RRT : 29 ms
End to end latency of this system : 14 ms

Total time RRT : 29 ms
End to end latency of this system : 14 ms

Total time RRT : 42 ms
End to end latency of this system : 21 ms

Total time RRT : 128 ms
End to end latency of this system : 64 ms

Total time RRT : 260 ms
End to end latency of this system : 130 ms

Total time RRT : 30 ms
End to end latency of this system : 15 ms
```

The average RTT is ~67 ms, corresponding to an average end-to-end latency of ~33 ms. Most measurements cluster between 14-25 ms, which is consistent with a local WiFi + MQTT broker setup. The two outliers at 64 ms and 130 ms are likely caused by FreeRTOS scheduler jitter, WiFi beacon sleep intervals, or temporary network congestion on the local network.

Per-Window Execution Time Measurement

Window time : ~ 5472 ms
Sampling time ~4953 instead of 5000ms

The total per-window execution time, measured from the start of the sampling phase until the MQTT publish call, was ~ 5472 ms ms. In particular, the sampling phase alone took ~4953 ms, slightly below the theoretical expected value of 5000 ms. This small negative deviation (~47 ms) can be attributed to integer rounding in the computation of nrSamples = ceil(sampleFreq * WINDOW_TIME_SAMPLING_SECONDS) and xFrequency = pdMS_TO_TICKS(1000 / sample_freq), which introduce truncation errors that accumulate over the entire window. The remaining time beyond the sampling phase is attributable to the FreeRTOS queue transfer latency between TaskSampling and TaskClientMQTT, the PubSubClient::loop() polling overhead, and the MQTT publish latency over WiFi.

Network Data Volume: Adaptive vs Over-Sampled Sampling Frequency

Over-sampled Case (1.8 MHz)

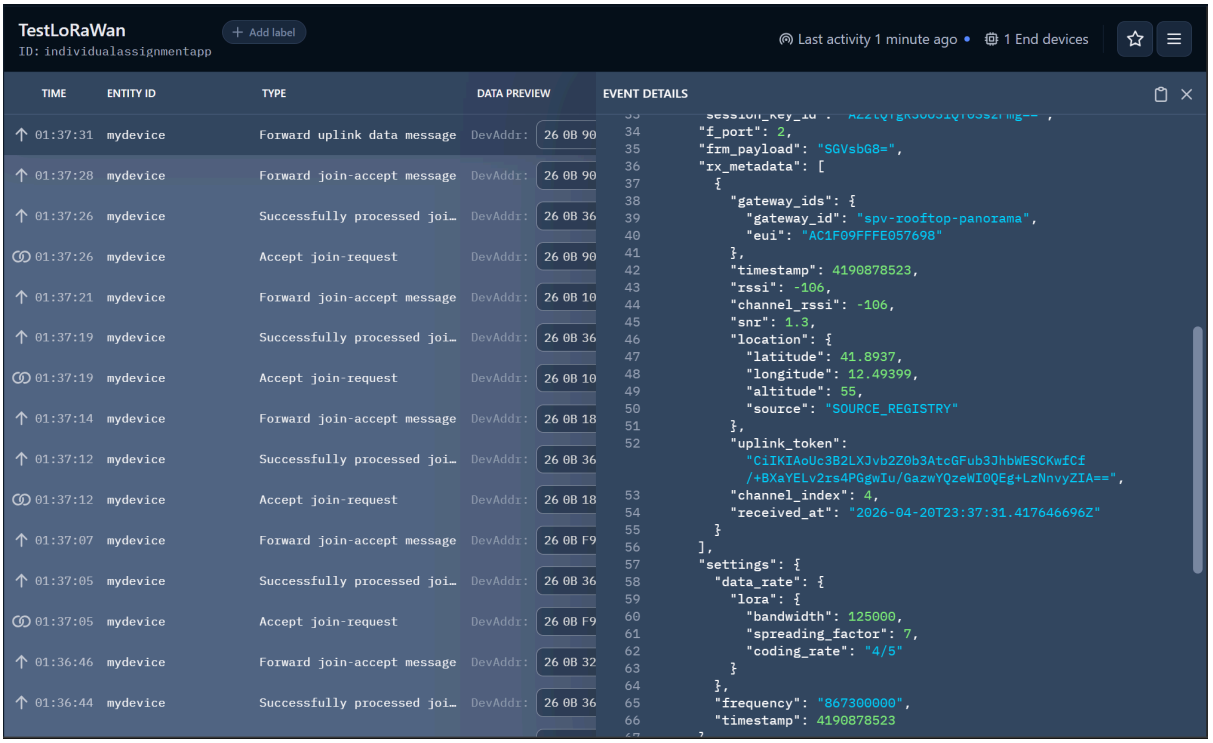
- **nrSamples = ceil(1.800.000 * 5) = 9.000.000 samples for window**
- Window lasts 5 seconds
- 1 MQTT message every 5 seconds
- Payload ≈ 40 bytes
- Volume: 40 / 5s = 8 bytes/s

Adaptive Case (13Hz)

- nrSamples = ceil(13 * 5) = 65 samples for window
- Window lasts 5 seconds
- 1 MQTT message every 5 seconds
- Payload ≈ 40 bytes
- Volume: 40 / 5s = 8 bytes/s

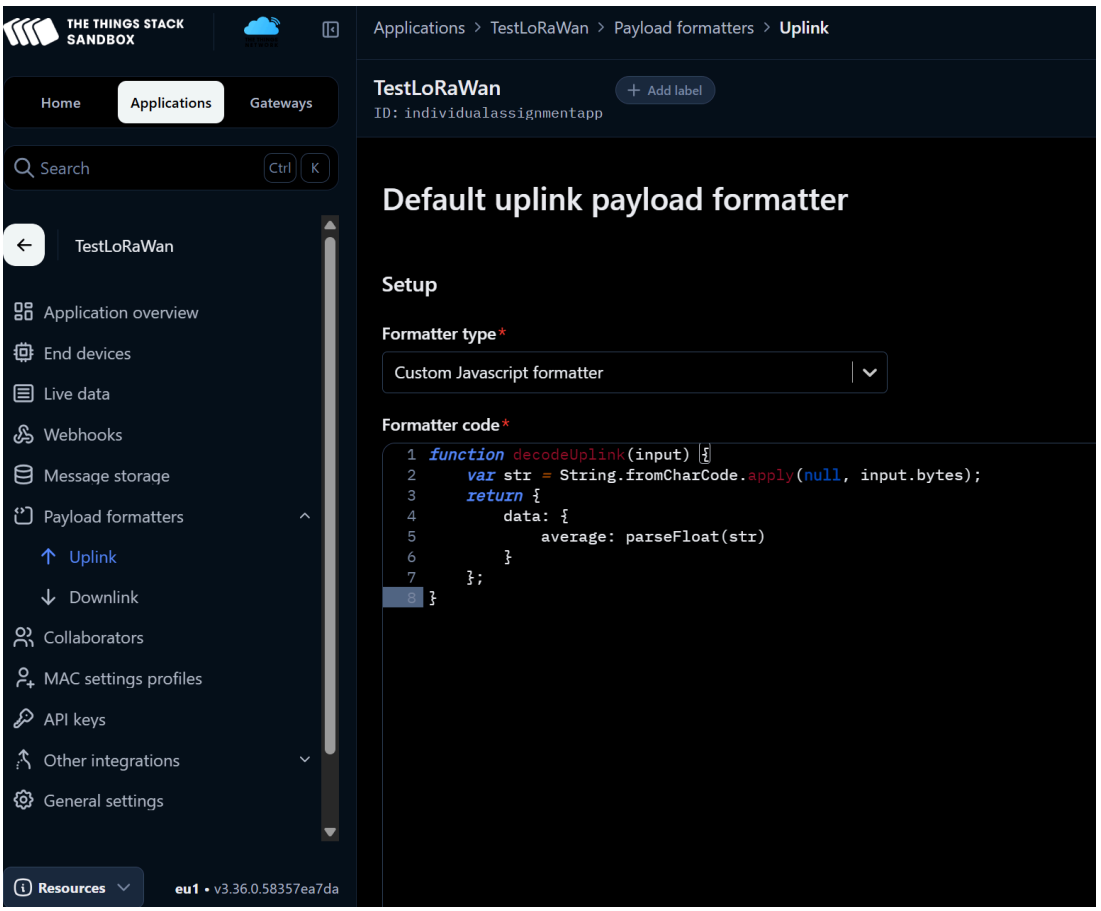
In both the over-sampled case (1.8 MHz) and the adaptive case (13 Hz), the network data volume is identical at 8 bytes/s, since the ESP32 only transmits the computed window average via a single MQTT message every 5 seconds, regardless of how many raw samples were collected internally. The reduction in sampling frequency from 1.8 MHz to 13 Hz therefore brings no benefit in terms of network traffic, but exclusively in terms of energy consumption and CPU load.

LoRa WAN + TTN

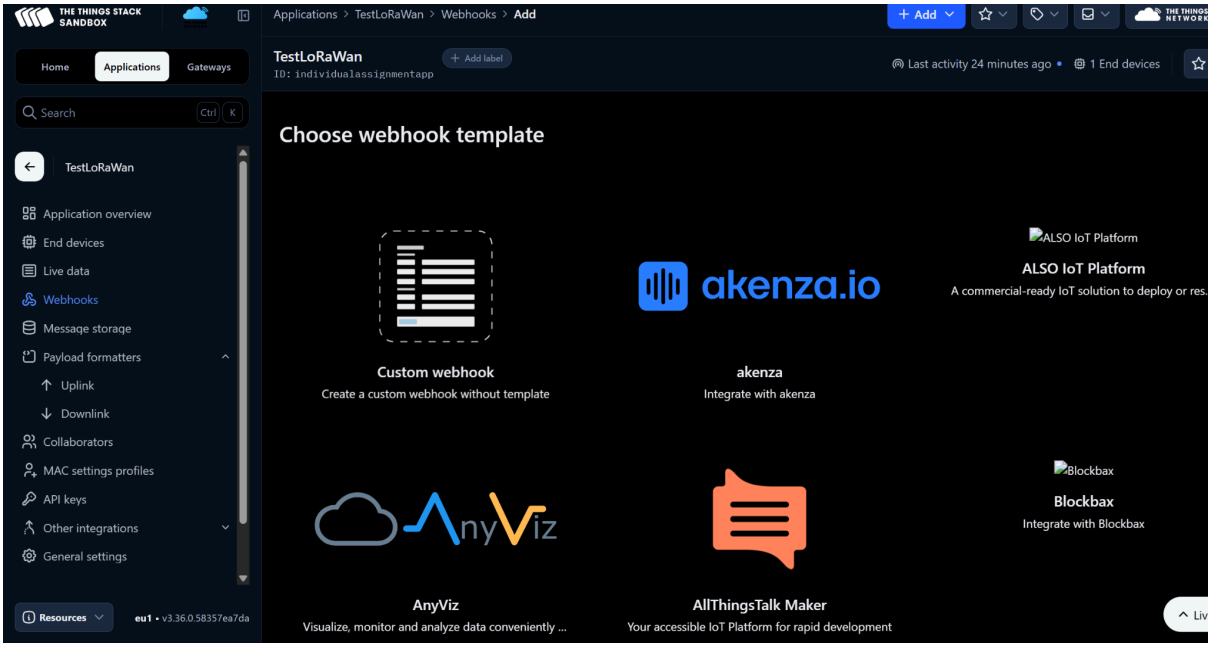


To forward the data from TTN to a cloud server, three steps were performed:

- **Payload Formatter**, A custom JavaScript function was written in TTN Console under Payload Formatters Uplink, which decodes the raw bytes received from the device into a human-readable float value, making the average field available as structured JSON in all downstream integrations.

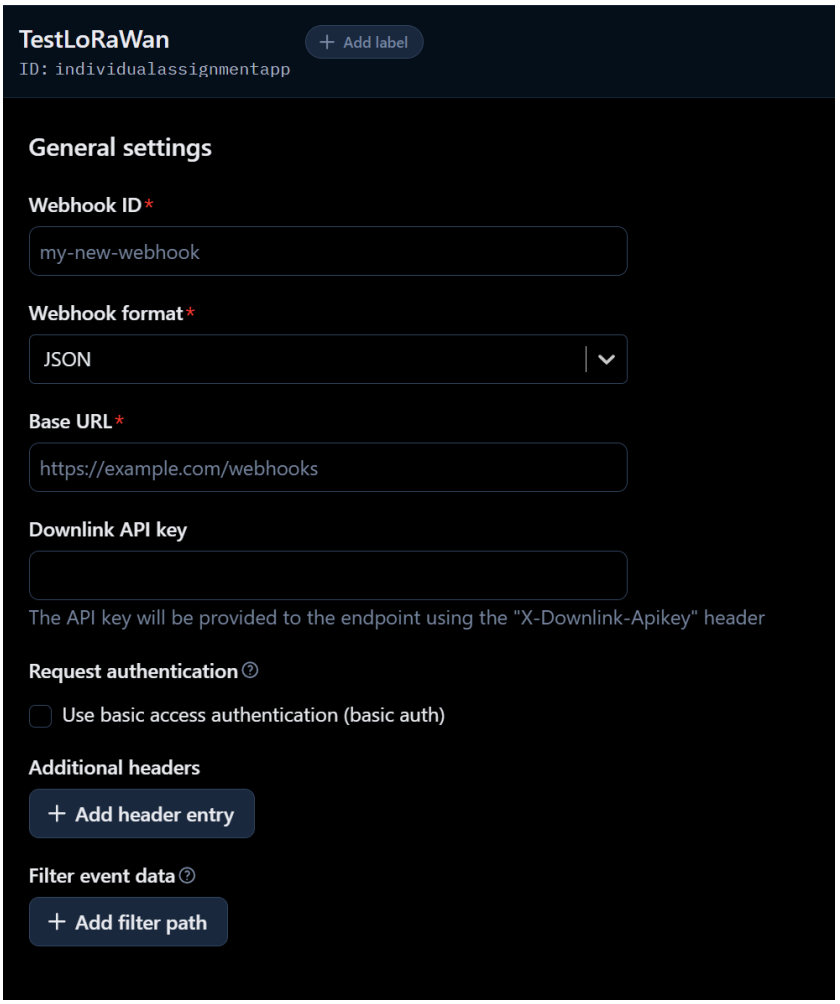


- **Webhook Integration**, In TTN Console under Integrations Webhooks, a new webhook was created by selecting Custom Webhook. This allows TTN to automatically forward every received uplink message to an external server.



- **Cloud Server URL**, The URL of the target cloud server was configured in the webhook settings. Every time the device transmits a new average value, TTN sends an HTTP POST request to that URL, with a JSON body containing the decoded

payload, device metadata, and timestamp. The receiving server can then store, process, or visualize the data as needed.



LLM:

LLM opportunities :

- LLMs are particularly convenient for debugging, since you can paste code directly and get explanations of what might be going wrong.

- They are also useful for understanding unfamiliar libraries given them the code, you can ask targeted questions and get working examples quickly.

- More generally, they are a good source of inspiration when you need a starting point or want to explore different approaches to a problem.

LLM limitations:

Using an LLM to write code has several notable limitations.

- LLMs have limited knowledge of specific library versions and microcontroller SDK constraints. API behavior, deprecated functions, and platform-specific quirks are often not reflected accurately in the generated code, requiring the developer to verify every suggestion against the official documentation.
- LLMs tend to generate generic solutions that may not be optimal for resource-constrained environments. On a microcontroller, stack size, heap allocation, and task priorities matter significantly, and an LLM may overlook these aspects without explicit prompting.
- While LLMs can speed up development considerably, they risk becoming a crutch that prevents deeper understanding. Relying on generated code without fully digesting it can lead to a fragile codebase where the developer loses track of the overall logic, making the project harder to maintain and debug over time.

In summary, LLMs are valuable for accelerating development and explaining concepts, but they must be treated as a starting point rather than a final solution. Critical thinking, hardware testing, and domain knowledge remain essential.

The code I wrote for this assignment was developed by taking inspiration from the sample code provided in the course GitHub repository, using it as a reference to better understand the logic and the steps involved. Starting from those examples, I intentionally built my own implementation, using LLMs mainly to clarify how specific library functions worked.

I used LLMs primarily as a debugging aid whenever I encountered issues such as buffer overflows or other bugs, I would ask for help understanding what type of problem it was and where it might originate. However, in most cases the LLM was not able to directly solve the issue, and the actual fix came from manual debugging and identifying the exact point in the code causing the problem.